BRUCEKIT

TECHNICAL SPECIFICATION · MODULAR TOOL LAUNCHER

- **Doc ID:** BRUCEKIT-SPEC-001
- **Date:** 2026-07-19
- **Status:** Approved design → ready for implementation plan
- **Owner:** bruce

§01 OVERVIEW

brucekit is a lightweight Windows desktop app: a single popup, triggered by a global hotkey, that shows a searchable grid of small self-contained utility tools — like the Start menu, but exclusively for tiny personal quality-of-life utilities. It is a growing personal library of little apps that all live in one place.

It runs in the system tray, launches on startup (optional), and is built to be **modular and extensible from day one**: adding a new tool is a folder drop-in with zero edits to unrelated tools.

Ships with two tools: **OCR grab** and **Color picker**.

» Goals

1. A fast, clean, dark HUD-styled popup opened by a global hotkey.
2. Searchable grid of tool tiles; select to launch a tool inline or via a native flow.
3. A clear tool-module contract so future tools slot in trivially.
4. Strong isolation: one misbehaving tool never affects the launcher or siblings.
5. Everything local — no backend, no database, no cloud. Config in a local file.

» Non-goals (YAGNI)

- No public/external plugin API. Modules are hardcoded in the repo by the owner.
- No process-level sandboxing of modules (they are trusted first-party code).
- No cloud OCR, telemetry, accounts, or sync.
- No cross-platform support in v1 (Windows-first; the OCR engine is abstracted so a cross-platform backend can be added later without touching tool code).

§02 STACK & DEPENDENCIES

- **Tauri v2** + **React** + **TypeScript** + **Vite**.
- Rust core owns all native work; React owns all UI. They communicate over a small, typed IPC command surface (§13).

Tauri plugins

Plugin	Purpose
tauri-plugin-global-shortcut	Register the global hotkey
tauri-plugin-autostart	Launch on login (toggleable)
tauri-plugin-store	Persist config.json in the app config dir
tauri-plugin-clipboard-manager	Copy OCR text / color values

Rust crates

Crate	Purpose
xcap	Cross-platform screen capture (per-monitor RGBA frames)
windows	Windows.Media.Ocr + SoftwareBitmap (native OCR)
image	Crop / convert captured frames
serde, serde_json	Config + IPC payload (de)serialization
thiserror	Typed error enums surfaced to the UI

§03 WINDOW MODEL

Two frameless windows, both rendered by the same Vite app and routed by window label (App.tsx reads getCurrentWindow().label).

» 3.1 Launcher (`label: "launcher"`)

- Frameless, transparent, always-on-top, skipTaskbar, no focus-steal on show.
- Hidden by default. Shown on hotkey; positioned centered on the active monitor.
- Dismissed on Esc or on blur (click-away).
- Contents: search box + tool grid (§6). Selecting a tool either runs its native flow (action kind) or expands an inline panel (panel kind).

» 3.2 Overlay (`label: "overlay"`)

- On-demand, fullscreen, transparent, always-on-top window used by any tool that needs the screen (OCR grab, Color picker). Created/shown on demand, hidden after.

» 3.3 Freeze-frame capture

When a screen tool activates:

1. Rust captures the active monitor into memory **first** (xcap).
2. The overlay is shown with that frozen snapshot as its backdrop.
3. The user drags a rectangle (OCR) or clicks a pixel (color) on the *static* image.
4. React sends the rect/point to Rust → crop / pixel-read → OCR or format → clipboard → toast.

Freezing the frame removes flicker, guarantees exact pixels, and makes the eyedropper a trivial pixel lookup. v1 targets the **monitor under the cursor**; multi-monitor compositing is a future enhancement.

§04 THE TOOL CONTRACT (CORE)

Every tool is a self-contained module exporting exactly one ToolModule as its default export.

```
// src/tools/types.ts

export type ToolContext = {
  invoke: TauriInvoke;                      // typed wrapper over tauri invoke
  toast: (msg: string, opts?: ToastOptions) =&gt; void;
  closeLauncher: () =&gt; void;
  settings: ToolSettings;                    // namespaced get/set for THIS tool
};

export type ToolKind = "action" | "panel";

export interface ToolModule {
  id: string;                                // stable unique id, e.g. "ocr-grab"
  name: string;                              // display name
  description?: string;                      // subtitle + search text
  keywords?: string[];                       // extra search terms
  icon: React.FC&lt;{ size?: number }&gt;;         // monochrome SVG icon component
  kind: ToolKind;
  activate?: (ctx: ToolContext) =&gt; void | Promise&lt;void&gt;;   // action tools
  render?: (ctx: ToolContext) =&gt; React.ReactNode;          // panel tools
}
```

» Tool kinds

- `action` — runs a native flow, no inline UI. Selecting the tile closes the launcher and runs `activate(ctx)`. → **OCR grab**.
- `panel` — renders inline UI inside the launcher via `render(ctx)`; the launcher expands to host it. → **Color picker**.

Shipping one of each proves both paths with the two starter tools.

Contract invariants

- `id` is unique and stable; the registry rejects duplicates (logs + skips).
- A `panel` tool must define `render`; an `action` tool must define `activate`. Malformed modules are skipped at registration, not at click time.
- `settings` is automatically namespaced by `id`, so tools cannot read/clobber each other's config.

§05 TOOL REGISTRY & AUTO-REGISTRATION

`src/tools/registry.ts` discovers tools with Vite `import.meta.glob`:

```
const modules = import.meta.glob("./*/index.tsx", { eager: true });
```

Each `tools/<name>/index.tsx` default-exports a `ToolModule`. **Dropping in a new `tools/<name>/` folder registers it automatically — zero edits to any other file.**

Registration is defensive: each module is validated (has `id`, `name`, `icon`, correct handler for its `kind`) inside a `try/catch`. A broken or duplicate module is logged to the console and skipped; all valid siblings still load.

The registry exposes `getTools()` and a `search(query)` helper that filters on `name`, `description`, and `keywords`.

§06 LAUNCHER UI

- **SearchBar** — autofocused on open; filters tiles live; `Enter` launches the top hit; arrow keys move selection; `Esc` closes.
- **ToolGrid / ToolTile** — monochrome icon + name; keyboard + mouse selectable.

- **ToolHost** — when a panel tool is selected, the grid collapses and the panel expands in place inside an **error boundary** (§7). A back affordance returns to the grid.
- Gear affordance opens **Settings** (§8).

§07 CRASH ISOLATION

The guarantee: a bug in one tool degrades gracefully and never takes down the launcher or sibling tools.

- **UI layer** — each tool's render() output is wrapped in a React **error boundary**. A render/runtime crash shows a compact fallback *inside that tool's panel only*; the launcher and other tools are unaffected.
- **Native layer** — OCR/capture are async Rust commands returning Result. Failures reject the promise and surface as a toast; the process never crashes.
- **Dispatch** — activate() / render() invocations are wrapped in try/catch; a throwing tool produces a toast + logged error, nothing more.

§08 CONFIG & SETTINGS

Persisted via tauri-plugin-store to config.json in the app config dir.

```
{
  "hotkey": "CommandOrControl+Shift+`",  // default; reconfigurable
  "launchOnStartup": false,
  "tools": {
    "color-picker": { "format": "hex" }   // per-tool namespace
  }
}
```

Settings panel (reachable from the launcher gear and the tray menu):

- Hotkey editor (capture a new chord; validates + re-registers).
- Launch-on-startup toggle (wired to autostart).
- Tool list (name + description of each registered tool).

§09 TRAY & STARTUP

- Tray icon with menu: **Open brucekit · Settings · Launch on startup (checkbox) · Quit**.
  Left-clicking the tray icon toggles the launcher.
- Closing/dismissing the launcher hides it (app keeps running in the tray).
- Launch-on-startup is off by default; toggling it wires autostart.

§10 STARTER TOOL — OCR GRAB (`ACTION`)

Flow: select tile → launcher closes → Rust freezes the active monitor → overlay shows the frozen frame with a crosshair → user drags a rectangle → React sends the rect → Rust crops the frame → OCR → text copied to clipboard → toast ("Copied N characters") → overlay hides.

Rust OCR behind a trait (so a cross-platform engine can be added later):

```
pub trait OcrEngine {
    fn recognize(&amp;self, img: &amp;image::RgbaImage) -&gt; Result&lt;String, OcrError&gt;;
}

pub struct WindowsOcr;   // Windows.Media.Ocr via the `windows` crate
```

`WindowsOcr` builds a `SoftwareBitmap` (BGRA8) from the cropped frame, creates an `OcrEngine` from the user's profile languages (fallback: `en`), runs `RecognizeAsync`, and returns `OcrResult.Text()`.

§11 STARTER TOOL — COLOR PICKER (`PANEL`)

Panel UI (inline in the launcher): a **HEX / RGB / HSL** format toggle (persisted to `tools.color-picker.format`), a large **swatch + value** of the last-picked color, a **Copy** button, and a **Pick pixel** button.

Pick flow: "Pick pixel" → launcher closes → Rust freezes the active monitor → overlay shows the frozen frame with a **magnifier loupe** + live pixel readout → user clicks a pixel → Rust reads that pixel from the frozen frame → color returned to the panel, copied to clipboard in the selected format, and shown in the swatch + toast.

Formatting is pure logic (unit-tested both sides):

- HEX — `#RRGGBB`
- RGB — `rgb(r, g, b)`
- HSL — `hsl(h, s%, l%)`

§12 AESTHETIC — BLACK-AND-WHITE TECHNICAL HUD

Matches brucelsprouts.com: terminal / sci-fi interface minimalism.

- Monospace type; near-black background; light-gray/white foreground.
- Monochrome palette + a single restrained signal accent.
- Hairline (1px) borders; corner tick / bracket framing on panels; subtle grid / scanline texture; small uppercase technical labels and coordinates.
- Fast, snappy transitions; no heavy chrome.

Design tokens live in `src/core/tokens.css` and are pulled from the live site when the CSS is authored.

§13 IPC COMMAND SURFACE (RUST ↔ JS)

Small and typed (`src/core/ipc.ts` wraps `invoke`):

Command	In	Out
`capture_monitor`	(cursor pos)	frame handle / dimensions
`ocr_region`	rect	`String` (recognized text)
`pick_color`	point	`{ r, g, b }`
`get_config` / `set_config`	key/value	config JSON
`set_hotkey`	chord	ok / error
`set_autostart`	bool	ok

Native flows are `async` and return `Result`; errors become typed rejections surfaced as toasts.

§14 FOLDER STRUCTURE

```
brucekit/
  src-tauri/
    src/
      lib.rs
      window.rs                # launcher + overlay window management
      hotkey.rs                # global shortcut register/re-register
      tray.rs                  # tray icon + menu
      commands/
        capture.rs             # xcap freeze-frame
        ocr.rs                 # OcrEngine trait + WindowsOcr
        color.rs               # pixel read + format
        config.rs              # store load/save
    tauri.conf.json
    Cargo.toml
    icons/
  src/
    main.tsx
    App.tsx                    # routes by window label
    launcher/
      Launcher.tsx  ToolGrid.tsx  ToolTile.tsx
      SearchBar.tsx  ToolHost.tsx  ErrorBoundary.tsx
    overlay/
      Overlay.tsx  RegionSelect.tsx  Eyedropper.tsx
    tools/
      types.ts  registry.ts
      ocr-grab/      index.tsx  icon.tsx
      color-picker/  index.tsx  icon.tsx  ColorPanel.tsx
    core/
      ipc.ts  toast.tsx  settings.ts  tokens.css
    components/  Toast.tsx  Settings.tsx
    styles/  global.css
  package.json  vite.config.ts  tsconfig.json  README.md
```

§15 TESTING

- **Vitest (TS):** registry load / dedupe / search; color conversions (hex/rgb/hsl); toast reducer; settings namespacing.
- **Rust unit tests:** color formatting; crop-bounds math; config round-trip.
- Native capture/OCR shells kept thin; pure logic carries the test coverage.

§16 BUILD & RUN

- `npm install`, `npm run tauri dev` for development.
- `npm run tauri build` for a Windows installer.

§17 FUTURE (OUT OF SCOPE FOR V1)

- Multi-monitor freeze-frame compositing.
- Cross-platform OCR backend (Tesseract) behind the existing `OcrEngine` trait.
- More tools (JSON formatter, unit converter, etc.) via folder drop-in.